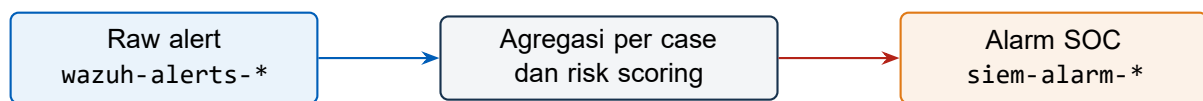


LOGIKA AGREGASI SIEM ALARM

untuk Wazuh 4.14.7 All-in-One



Arsitektur, alur data, rumus, contoh log, reliabilitas,
keamanan, operasi, dan batas penggunaan nyata

Baseline: Wazuh 4.14.7 AIO / single node
Bucket: 60 menit, UTC
Jadwal: setiap 5 menit
Dokumen keluaran: alarm_state dan alarm_escalation
Tanggal dokumen: 26 Agustus 2026

Dokumen ini menjelaskan perilaku implementasi pada `siem_alarm_scoring_final.py`. Ia bukan pengganti hasil load test dan validasi pada server produksi tertentu.

Daftar Isi

1	Ringkasan Eksekutif	1
2	Tujuan dan Lingkup Desain	2
3	Komponen dan Berkas Proyek	3
4	Topologi Wazuh All-in-One dengan Agregasi	4
5	Flowchart Proses Satu Run	5
6	Model Waktu dan Bucket	7
7	Definisi “Alert yang Sama”	8
8	Agregasi Source IP dan Evidence	9
9	Field Extraction dan Normalisasi	10
10	Asset Value dan Sumber Kepercayaan	11
11	Threat Score dari Wazuh Rule Level	12
12	Frequency Score	13
13	Rumus dan Level Risiko	14
14	Dataset Dummy Rule Default Wazuh dan Hasil Proses	15
14.1	Status dan batas contoh	15
14.2	Satu raw alert dalam format representatif	15
14.3	Sepuluh raw alert dalam satu bucket	16
14.4	Tahap 1: pembentukan satu case	17
14.5	Tahap 2: snapshot pertama setelah lima alert	17
14.6	Tahap 3: snapshot kedua setelah sepuluh alert	18
14.7	Tahap 4: log alarm eskalasi	19
14.8	Apa yang terjadi pada source IP berbeda?	19
15	Contoh End-to-End: Failed Login	21
16	Dokumen alarm_state	23
17	Dokumen alarm_escalation	25

18 Lifecycle State dan Riwayat Level	26
19 Two-Phase Bulk dan Idempotensi	27
20 Checkpoint, Catch-Up, dan Late Alert	28
21 Fail-Closed dan Guardrail	29
22 Keamanan dan Least Privilege	31
23 Performa dan Kapasitas	32
24 Dashboard dan Query Operasional	33
25 Validasi Count terhadap Raw Evidence	34
26 Instalasi dan Urutan Go-Live	35
27 Monitoring Harian dan Troubleshooting	36
28 Batas Desain dan Tuning	38
29 Checklist Penerimaan Produksi	39
30 Kesimpulan	40
A Ringkasan Konfigurasi Baseline	41
B Matriks Field Output	42
C Glosarium	43
D Referensi Teknis	44

Daftar Gambar

4.1	Topologi logis Wazuh AIO setelah sistem agregasi diterapkan.	4
5.1	Flowchart satu eksekusi engine agregasi.	6
6.1	Lifecycle waktu satu bucket baseline.	7
14.1	Pemrosesan dataset dummy oleh engine agregasi proyek.	17
19.1	Urutan penulisan agar event tidak tertinggal di belakang state.	27

Daftar Tabel

7.1	Mode deduplikasi yang tersedia.	8
10.1	Skala Asset Value.	11
11.1	Konversi Wazuh rule level menjadi Threat Score.	12
12.1	Konversi raw alert count menjadi Frequency Score.	13
13.1	Konversi Risk Score menjadi Risk Level.	14
14.1	Dataset dummy rule 5710 dalam bucket 10:00–11:00 UTC.	17
15.1	Perkembangan progressive alarm.	21
22.1	Hak minimum logis runtime.	31

Bab 1

Ringkasan Eksekutif

Sistem agregasi SIEM Alarm dibuat untuk mengurangi jumlah alert mentah yang harus dibaca analis tanpa menghapus bukti asalnya. Wazuh tetap menjadi mesin deteksi. Setiap dokumen Wazuh yang cocok tetap disimpan pada `wazuh-alerts-*`; sistem tambahan ini membaca dokumen tersebut, menyatukan alert yang dianggap satu kasus, memberikan skor risiko, lalu menulis ringkasannya ke `siem-alarm-*`.

Definisi default satu kasus adalah kombinasi `agent.id`, `rule.id`, dan bucket waktu satu jam dalam UTC. Source IP, destination IP, port, user, URL, dan file tidak hilang. Pada mode default semuanya menjadi evidence di dalam satu alarm, bukan pemecah alarm. Karena itu serangan dari botnet atau rotating source IP tidak serta-merta menghasilkan ratusan kartu alarm terpisah.

Sistem menghasilkan dua tipe dokumen. `alarm_state` adalah keadaan terbaru dari satu kasus dan ditulis ulang menggunakan document ID yang sama. `alarm_escalation` adalah catatan immutable ketika risiko pertama kali masuk level yang dipantau atau naik ke level lebih tinggi. Pemisahan ini membuat dashboard dapat menampilkan keadaan terakhir, sementara aplikasi eksternal dapat mengonsumsi stream perubahan risiko.

Pokok penting. Status teknis paket adalah siap untuk staging atau shadow run pada baseline yang ditentukan. Status produksi baru dapat diberikan setelah field audit, validasi TLS dan role, rekonsiliasi count dengan raw alert, pengukuran lag, serta load test pada host Wazuh yang sebenarnya selesai.

Bab 2

Tujuan dan Lingkup Desain

Tujuan utama sistem bukan mengganti korelasi Wazuh, melainkan membentuk lapisan prioritas operasional untuk SOC. Wazuh menentukan bahwa sebuah event adalah alert melalui decoder dan rule. Engine agregasi baru bekerja setelah keputusan tersebut dibuat.

Baseline yang menjadi sasaran adalah:

- Ubuntu dengan instalasi Wazuh all-in-one;
- Wazuh Manager, Wazuh Indexer, dan Wazuh Dashboard versi 4.14.7;
- Filebeat OSS 7.10.2;
- satu node Indexer, sehingga template memakai satu primary shard dan nol replica;
- sekitar 5 sampai 25 agent, dengan kapasitas aktual ditentukan EPS dan cardinality;
- eksekusi serial setiap lima menit melalui systemd timer;
- bucket tetap 60 menit.

Yang tidak otomatis tercakup adalah deployment distributed, multi-node Indexer, container, Wazuh Cloud, source index dengan pembagian tanggal non-UTC, serta workflow incident management seperti acknowledge, assignment, atau resolved. Field `alarm.status` hanya menyatakan lifecycle bucket: `open` atau `finalized`.

Sistem juga bukan UEBA, bukan machine learning, dan bukan pengganti penilaian risiko organisasi. Skor yang dihasilkan adalah heuristik deterministik agar prioritas dapat dijelaskan dan diaudit.

Bab 3

Komponen dan Berkas Proyek

Implementasi dibagi menjadi berkas yang memiliki tanggung jawab jelas.

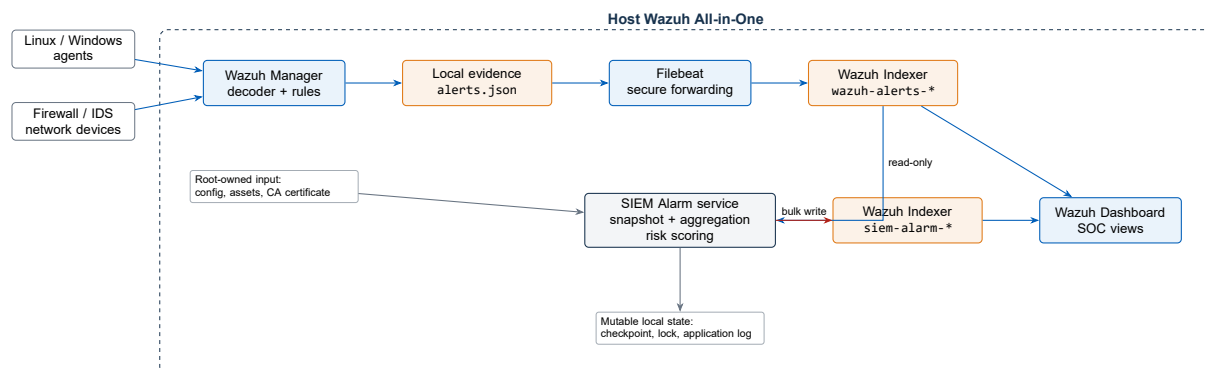
Berkas	Fungsi
siem_alarm_scoring_final.py	Engine query, agregasi, scoring, checkpoint, <code>_mget</code> , dan <code>two-phase_bulk</code> .
wazuh_field_audit_final.py	Mengambil sampel raw alert dan melaporkan field yang benar-benar ada.
config.siem_alarm.example.json	Baseline konfigurasi runtime dan seluruh guardrail.
assets.example.json	Contoh schema nilai aset; bukan inventory production.
siem_alarm_template_final.json	Mapping dan setting untuk index hasil.
siem_alarm_ism_policy.json	Retensi default 90 hari untuk index hasil baru.
setup_siem_alarm_final.sh	Preflight, backup, instalasi, user Linux, systemd, dan logrotate.
tests/	Regression test untuk fungsi inti, retry, idempotensi, cap, dan checkpoint.

Konfigurasi dan inventory aset dipasang root-owned tetapi dapat dibaca group service. Secret tidak disimpan di JSON; password dibaca dari environment file systemd. Mutable checkpoint dipisahkan ke `/var/lib/wazuh-risk-scoring`, sedangkan log ditulis ke direktori aplikasi yang secara eksplisit diizinkan oleh sandbox systemd.

Bab 4

Topologi Wazuh All-in-One dengan Agregasi

Gambar berikut menunjukkan komponen logis pada satu host AIO. Panah biru adalah alur raw alert atau read, sedangkan panah merah adalah penulisan dokumen hasil.



Gambar 4.1: Topologi logis Wazuh AIO setelah sistem agregasi diterapkan.

Raw index dan alarm index dapat berada pada cluster Indexer yang sama, tetapi hak aksesnya berbeda. Runtime hanya membaca `wazuh-alerts-*`; ia tidak mengubah atau menghapus bukti asal. Pada `siem-alarm-*`, runtime memerlukan read untuk membaca state sebelumnya, index/create untuk menulis, dan create-index agar index harian pertama dapat dibuat setelah template terpasang.

Bab 5

Flowchart Proses Satu Run

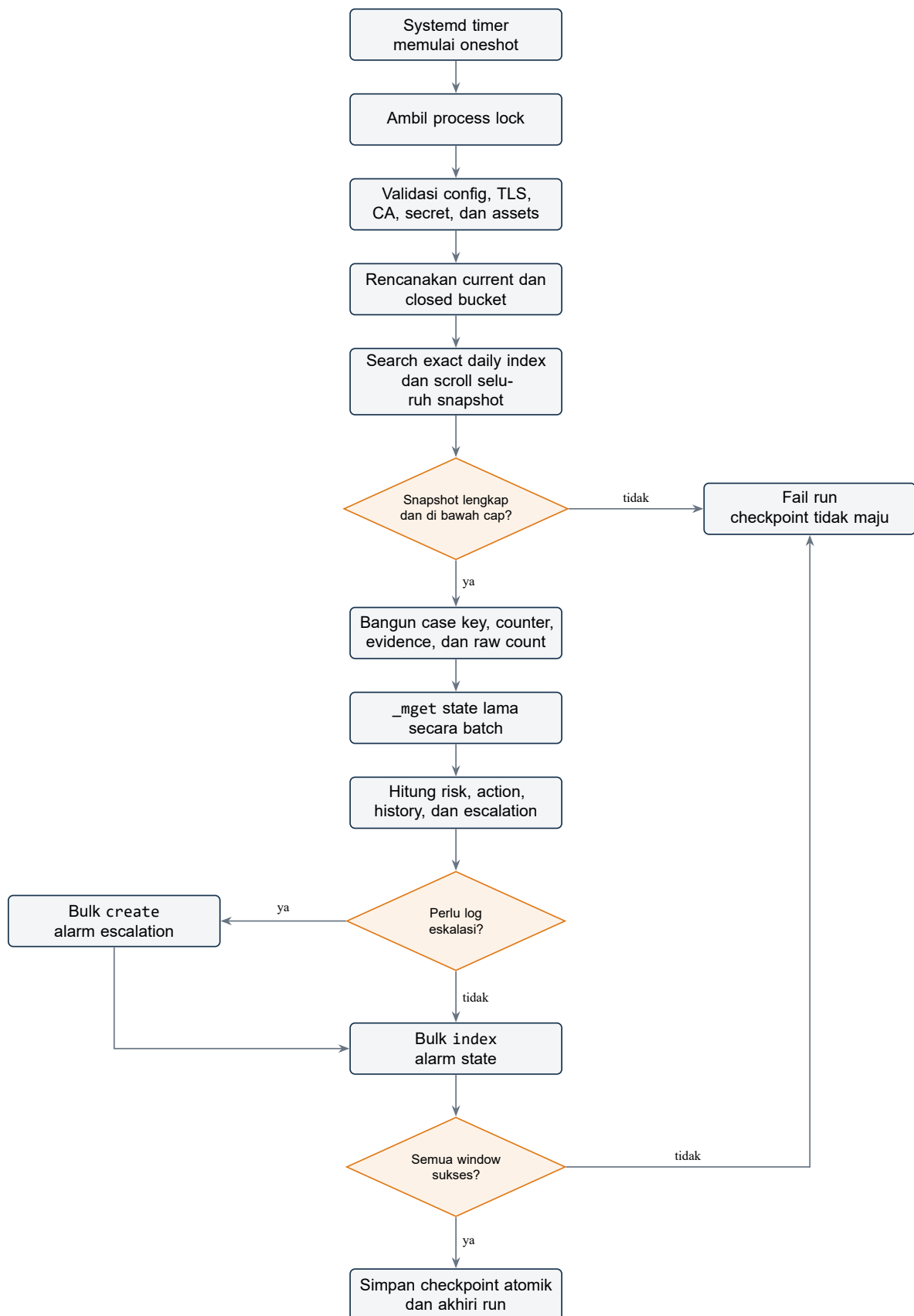
Satu run kalender bersifat bounded dan fail-closed. Kegagalan validasi, shard, scroll, bulk, atau cap menghentikan run dan mencegah checkpoint maju.

Flowchart pada halaman berikut dapat dibaca sebagai lima fase berurutan:

1. **preflight runtime**: lock, konfigurasi, TLS, secret, dan inventory;
2. **window planning**: memilih current bucket dan closed bucket yang eligible;
3. **snapshot acquisition**: membaca exact daily index sampai seluruh hit lengkap;
4. **materialization**: membentuk case, scoring, event escalation, dan state;
5. **commit recovery**: checkpoint hanya maju bila semua window sukses.

Cabang menuju *Fail run* bukan sekadar error handling. Cabang tersebut adalah kontrol integritas untuk memastikan dashboard tidak menerima snapshot yang diketahui tidak lengkap. Raw alert tetap tersedia, sehingga run dapat diulang setelah penyebab masalah diselesaikan.

Poin pentingnya adalah state tidak ditulis sebelum event eskalasi yang bergantung padanya berhasil dibuat. Jika bulk escalation gagal sebagian, state terkait ditahan. Run berikutnya aman mengulang karena event memakai create-only ID deterministik dan state memakai index ID deterministik.



Gambar 5.1: Flowchart satu eksekusi engine agregasi.

Bab 6

Model Waktu dan Bucket

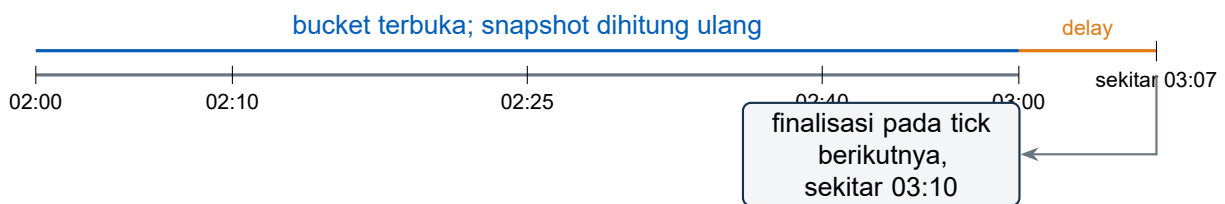
Semua identitas bucket dihitung dalam UTC. Timestamp Wazuh yang memakai bentuk +0000, -0500, atau offset lain dinormalisasi terlebih dahulu. Untuk konfigurasi baseline 60 menit, timestamp berikut semuanya masuk bucket yang sama:

```
2026-04-21T02:00:00Z
2026-04-21T02:42:05.483+0000
2026-04-21T09:58:33+0700
```

Ketiganya berada pada rentang setengah terbuka:

$$2026-04-21T02:00:00Z \leq t < 2026-04-21T03:00:00Z$$

Current bucket dihitung ulang dari boundary sampai waktu run. Bucket tertutup baru eligible setelah delay default tujuh menit. Dengan timer lima menit, finalisasi normal terjadi pada tick sekitar boundary ditambah sepuluh menit, bukan tepat tujuh menit.



Gambar 6.1: Lifecycle waktu satu bucket baseline.

Alert yang baru masuk Indexer setelah bucket difinalisasi tidak direvisi otomatis. Ia memerlukan backfill eksplisit. Karena itu delay harus lebih besar daripada lag end-to-end normal dan p99 lag perlu dimonitor.

Bab 7

Definisi “Alert yang Sama”

Mode default coarse menghasilkan key:

```
coarse|<agent.id>|<rule.id>|<bucket_start_UTC>
```

Dengan demikian dua source IP, dua username, atau dua URL berbeda tetap menjadi satu alarm ketika agent, rule, dan bucket sama. Ini cocok untuk suppression dan pengelompokan attack wave, tetapi belum tentu cocok untuk seluruh rule.

Tabel 7.1: Mode deduplikasi yang tersedia.

Mode	Field identitas	Penggunaan yang disarankan
coarse	agent.id, rule.id, bucket	Default; noise suppression per agent dan rule.
target_aware	coarse + dstip	Sensor IDS/firewall yang mengamati banyak server tujuan.
target_port_aware	target + dstport + proto	Rule jaringan ketika layanan tujuan harus dipisahkan.
file_aware	coarse + file path	FIM/malware ketika setiap path harus menjadi alarm tersendiri.

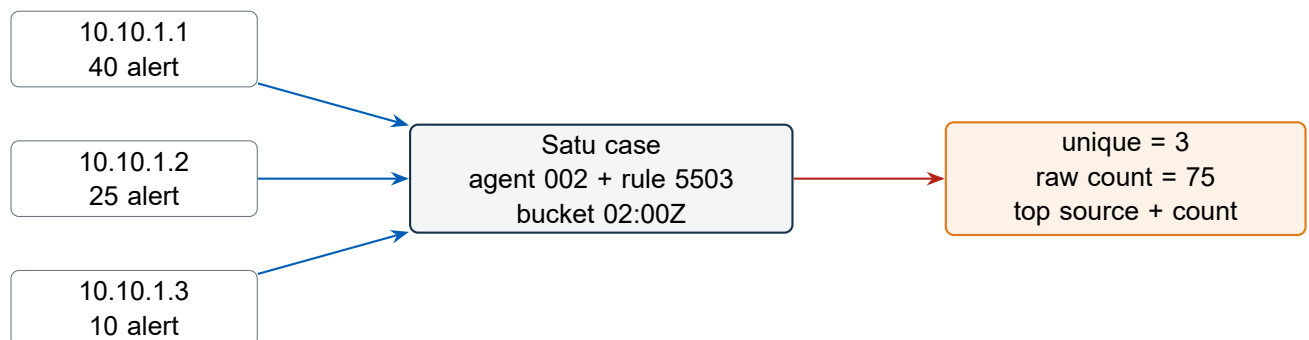
Mode dapat ditetapkan global dan dioverride per `rule.id`. Source IP sengaja tidak menjadi split key pada mode yang tersedia. Jika organisasi memerlukan alarm per source IP atau per username, implementasi perlu mode identitas tambahan dan migrasi destination/checkpoint yang terkontrol.

Perubahan mode mengubah identitas kasus. Engine menyimpan hash identitas konfigurasi di checkpoint dan akan fail-closed bila konfigurasi identitas berubah secara diam-diam.

Bab 8

Agregasi Source IP dan Evidence

Source IP yang berbeda tidak menggantikan satu sama lain. Engine menggunakan counter per nilai. Misalkan terdapat 75 raw alert:



Hasil ringkasannya:

```
"source_observed": {
  "srcip_unique_count": 3,
  "srcip_samples": ["10.10.1.1", "10.10.1.2", "10.10.1.3"],
  "top_srcip": [
    {"value": "10.10.1.1", "count": 40},
    {"value": "10.10.1.2", "count": 25},
    {"value": "10.10.1.3", "count": 10}
  ]
}
```

`srcip_unique_count` menghitung semua nilai unik yang dilihat snapshot. `srcip_samples` menyimpan maksimal 20 nilai dan `top_srcip` maksimal 10 pasangan nilai-count secara default. Bila terdapat 100 IP unik, count tetap 100, tetapi hanya nilai teratas yang dimaterialisasi ke alarm. Daftar diurutkan menurut frekuensi menurun lalu teks nilai, bukan menurut waktu kedatangan.

Nilai yang tidak masuk batas sampel tidak hilang dari sistem: raw document tetap berada pada `wazuh-alerts-*`. Batas tersebut hanya menjaga ukuran dokumen alarm.

Bab 9

Field Extraction dan Normalisasi

Engine mencari observed field melalui beberapa alias yang lazim ditemukan pada Wazuh dan ECS. Contoh source IP meliputi `data.srcip`, `data.src_ip`, `srcip`, `source.ip`, dan beberapa field Windows. Destination IP dapat jatuh kembali ke `agent.ip` bila tidak ada destination eksplisit.

Observed data yang diringkas mencakup:

- source IP;
- destination IP, port, dan protocol;
- URL dan user agent;
- username;
- file path dan hash;
- CVE dan SCA check ID.

Protocol angka umum dinormalisasi: 6 menjadi TCP, 17 menjadi UDP, dan 1 menjadi ICMP. Nilai kosong, tanda minus, `0.0.0.0`, dan `::` diperlakukan sebagai tidak terobservasi.

Field audit wajib dijalankan sebelum production karena decoder dan custom rule dapat menghasilkan path berbeda. Menambahkan path ke `source_includes` hanya membuat field ikut dikirim oleh Indexer. Jika nama tersebut bukan alias yang dikenali fungsi ekstraksi, kode alias juga harus diperluas dan diuji.

Bab 10

Asset Value dan Sumber Kepercayaan

Asset Value adalah komponen pertama skor. Urutan sumbernya:

1. `inventory assets.json`, lookup berdasarkan `agent.id`;
2. lookup berdasarkan `agent.name` jika ID tidak ditemukan;
3. label resmi `agent.labels.*` sebagai fallback;
4. nilai default 3 atau Medium disertai warning.

Tabel 10.1: Skala Asset Value.

Nilai	Kategori	Contoh
5	Critical	Database inti, AD/LDAP, sistem keselamatan, firewall kritikal.
4	High	Web server produksi, mail, VPN, sensor IDS penting.
3	Medium	Application server internal atau aset tanpa klasifikasi khusus.
2	Low	Workstation biasa, printer, endpoint berdampak rendah.
1	Minimal	Development/test yang terisolasi.

Inventory divalidasi secara ketat. Nilai harus integer 1 sampai 5, category harus sesuai, field tidak dikenal ditolak, dan mismatch nama untuk agent ID yang sama menghentikan run. Pemeriksaan mismatch mencegah nilai aset lama melekat diam-diam setelah agent dihapus dan didaftarkan ulang menggunakan ID yang sama.

Untuk production, `asset.source=default` pada agent wajib inventory harus dianggap masalah data, bukan keadaan normal.

Bab 11

Threat Score dari Wazuh Rule Level

Komponen kedua berasal dari `severity rule.level`. Strategi default adalah `max`. Engine juga menyimpan `mode`, `median`, `maximum`, dan distribusi level agar hasil tetap dapat diaudit apabila data suatu case ternyata bervariasi.

Tabel 11.1: Konversi Wazuh rule level menjadi Threat Score.

Wazuh rule.level	Threat Score
0–3	1
4–6	2
7–9	3
10–12	4
13–16	5

Strategi `max` konservatif terhadap satu alert severity tinggi. Strategi `mode` memilih level yang paling sering, dengan tie memilih level tertinggi. Strategi `median` memilih weighted median. Untuk rule Wazuh biasa, level umumnya statis per rule ID; ketiga statistik terutama menjadi guardrail dan bukti.

Threat Score tidak menilai confidence, MITRE technique, exploitability, atau konteks campaign. Organisasi yang membutuhkan komponen tersebut harus mengubah model risiko secara eksplisit, bukan menganggap level Wazuh sebagai probabilitas.

Bab 12

Frequency Score

Frequency adalah jumlah dokumen raw yang memiliki case key sama dalam satu bucket. Ia bukan jumlah source IP unik dan bukan `rule.firedtimes`.

Tabel 12.1: Konversi raw alert count menjadi Frequency Score.

Jumlah raw alert per satu jam	Frequency Score
1–9	1
10–49	2
50–99	3
100–499	4
500 atau lebih	5

Tiga field berikut dibentuk dari counter yang sama dan harus selalu identik:

```
alarm.event_count
risk.frequency_count_1h
source.raw_alert_count
```

Current bucket dihitung ulang dari raw evidence pada setiap run. Pendekatan snapshot ini lebih mahal daripada counter incremental, tetapi menghilangkan double count ketika request diulang, service crash, atau operator menjalankan ulang bucket.

Nama field dan threshold mengasumsikan bucket 60 menit. Walaupun validator menerima beberapa ukuran bucket lain yang membagi 1.440 menit, baseline production harus mempertahankan `bucket_minutes=60` sampai schema dan kebijakan frequency diubah secara konsisten.

Bab 13

Rumus dan Level Risiko

Skor risiko adalah rata-rata sederhana dari tiga komponen integer:

$$R = \frac{A + T + F}{3}$$

dengan A = Asset Value, T = Threat Score, dan F = Frequency Score. Hasil dibulatkan dua digit desimal sebelum ditulis.

Tabel 13.1: Konversi Risk Score menjadi Risk Level.

Rentang	Level	Tindakan default
$R < 1.50$	Information	Baseline; No SLA
$1.50 \leq R < 2.50$	Low	Monitor; Best effort
$2.50 \leq R < 3.50$	Medium	Triage; 1 business day
$3.50 \leq R < 4.50$	High	Investigate; 1 hour
$R \geq 4.50$	Critical	Immediate investigation; 15 minutes

Equal weighting membuat rumus mudah dijelaskan, tetapi juga berarti satu komponen tidak dapat mendominasi sendirian. Contoh asset Critical dengan rule level rendah dan frequency rendah menghasilkan $(5 + 1 + 1)/3 = 2.33$, yaitu Low. Bila kebijakan SOC mengharuskan semua alert pada aset tertentu minimal High, model saat ini harus ditambah rule override atau floor tersendiri.

Bab 14

Dataset Dummy Rule Default Wazuh dan Hasil Proses

Bab ini menyediakan contoh yang dapat direproduksi, bukan hanya ilustrasi lepas. Input lengkap berada pada `docs/examples/dummy_wazuh_alerts_rule_5710.json`; hasil yang diharapkan berada pada `docs/examples/dummy_aggregation_result_rule_5710.json`. Unit test `tests/test_documentation_examples.py` memuat kedua file itu dan menjalankan fungsi agregasi proyek secara langsung.

14.1 Status dan batas contoh

Semua identitas, waktu, hostname, username, document ID, dan log dalam dataset ini adalah **sintetis**. Dataset tidak berasal dari insiden atau host produksi. Alamat `203.0.113.0/24`, `198.51.100.0/24`, dan `192.0.2.0/24` merupakan rentang TEST-NET untuk dokumentasi, sehingga tidak menunjuk penyerang nyata.

Bentuk dokumen sengaja mengikuti hit OpenSearch pada index `wazuh-alerts-4.x-*`: `envelope_index/_id/_source`, lalu field `timestamp`, `agent`, `manager`, `input`, `full_log`, `predecoder`, `decoder`, `data`, `location`, dan `rule`. Field tambahan dapat berbeda menurut versi decoder, integrasi, dan konfigurasi Wazuh setempat; oleh sebab itu contoh ini adalah representasi schema nyata, bukan janji bahwa semua instalasi mempunyai field opsional yang identik.

Contoh memakai rule bawaan SSHD 5710, level 5, untuk percobaan login menggunakan user yang tidak ada. Identitas, level, serta relasinya dengan rule brute-force 5712 dicocokkan dengan ruleset Wazuh 4.14.7 [4].

Pokok penting. Raw dummy mewakili data yang *sudah* didekode dan sudah menjadi alert Wazuh. Engine agregasi tidak membaca baris `auth.log` langsung dan tidak menjalankan decoder/rule Wazuh ulang; engine membaca document hasil akhir dari `wazuh-alerts-4.x-2026.08.26`.

14.2 Satu raw alert dalam format representatif

Dokumen pertama dari sepuluh alert dummy adalah:

```
{
  "_index": "wazuh-alerts-4.x-2026.08.26",
  "_id": "dummy-5710-001",
```

```

"_source": {
  "timestamp": "2026-08-26T10:05:11.123+0000",
  "agent": {
    "id": "002",
    "name": "srv-web-01",
    "ip": "192.168.100.99"
  },
  "manager": {"name": "wazuh-aio"},
  "id": "1787748311.100001",
  "input": {"type": "log"},
  "full_log": "Aug 26 17:05:11 srv-web-01 sshd[24101]: Invalid user oracle from
203.0.113.10 port 51101",
  "predecoder": {
    "program_name": "sshd",
    "timestamp": "Aug 26 17:05:11",
    "hostname": "srv-web-01"
  },
  "decoder": {"parent": "sshd", "name": "sshd"},
  "data": {
    "srcip": "203.0.113.10",
    "srcport": "51101",
    "srcuser": "oracle"
  },
  "location": "/var/log/auth.log",
  "rule": {
    "id": "5710",
    "level": 5,
    "firedtimes": 1,
    "mail": false,
    "description": "sshd: Attempt to login using a non-existent user",
    "groups": [
      "syslog", "sshd", "invalid_login", "authentication_failed"
    ],
    "mitre": {
      "id": ["T1110"],
      "tactic": ["Credential Access"],
      "technique": ["Brute Force"]
    }
  }
}
}
}

```

Field yang dipakai engine dari dokumen tersebut adalah `timestamp`, `agent.id`, `agent.name`, `agent.ip`, `rule.id`, `rule.level`, `rule.description`, `rule.groups`, `data.srcip`, dan `data.srcuser`. Field seperti `full_log`, `predecoder`, serta `manager` tetap penting untuk forensik, tetapi tidak disalin penuh ke alarm agregat. Nilai tersebut tetap aman di raw index dan dapat ditemukan melalui pointer sample.

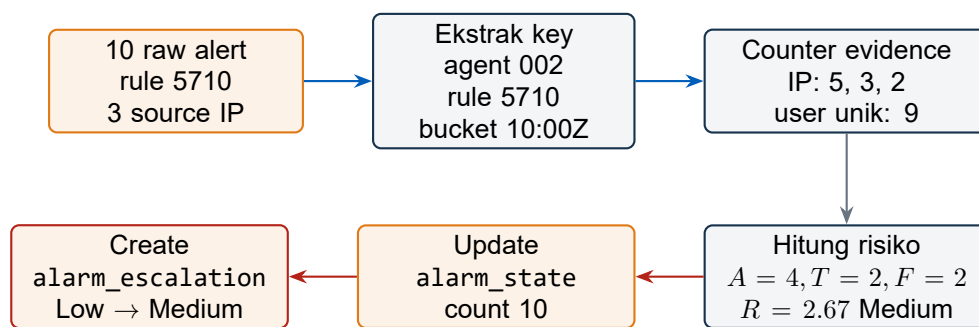
14.3 Sepuluh raw alert dalam satu bucket

Variasi sepuluh raw alert diringkas pada tabel berikut. Seluruh baris mempunyai agent 002, rule 5710, dan timestamp UTC dalam jam yang sama.

No.	Waktu UTC	data.srcip	data.srcuser
1	10:05:11	203.0.113.10	oracle
2	10:06:22	203.0.113.10	test

No.	Waktu UTC	data.srcip	data.srcuser
3	10:07:35	198.51.100.24	admin
4	10:08:47	203.0.113.10	postgres
5	10:10:02	192.0.2.44	deploy
6	10:11:18	198.51.100.24	guest
7	10:12:29	203.0.113.10	ubuntu
8	10:14:40	198.51.100.24	backup
9	10:16:51	203.0.113.10	support
10	10:18:03	192.0.2.44	admin

Tabel 14.1: Dataset dummy rule 5710 dalam bucket 10:00–11:00 UTC.



Gambar 14.1: Pemrosesan dataset dummy oleh engine agregasi proyek.

14.4 Tahap 1: pembentukan satu case

Mode default adalah *coarse*. Untuk setiap raw alert, engine membulatkan timestamp ke awal jam UTC dan membentuk:

```

case_key = coarse|002|5710|2026-08-26T10:00:00Z
state_id = SHA-256(case_key)
          = eccbdf4cc5236eff5993454b8c1ecd535e836e8bbab012d13bd196f9077bb347
  
```

Ketiga source IP **tidak menjadi komponen case key**. Karena agent, rule, dan bucket sama, seluruh sepuluh alert masuk ke satu case. Apabila Wazuh juga menghasilkan rule 5712, alert itu menjadi case lain karena `rule.id` berbeda, walaupun berasal dari raw SSHD yang berkaitan.

14.5 Tahap 2: snapshot pertama setelah lima alert

Pada run pertama baru lima raw alert terlihat. Counter menghasilkan tiga source IP unik dengan distribusi 3, 1, dan 1. Inventory memberi agent 002 Asset Value 4; rule level 5 memberi Threat Score 2; count 5 memberi Frequency Score 1.

$$R_5 = \frac{4 + 2 + 1}{3} = 2.33 \implies \text{Low}$$

Potongan state yang ditulis:

```
{
  "alarm": {"event_count": 5},
  "source_observed": {
    "srcip_unique_count": 3,
    "srcip_samples": [
      "203.0.113.10", "192.0.2.44", "198.51.100.24"
    ],
    "top_srcip": [
      {"value": "203.0.113.10", "count": 3},
      {"value": "192.0.2.44", "count": 1},
      {"value": "198.51.100.24", "count": 1}
    ]
  },
  "risk": {
    "asset_value": 4,
    "threat_score": 2,
    "frequency_count_1h": 5,
    "frequency_score": 1,
    "score": 2.33,
    "level": "Low",
    "escalation_log_required": false
  }
}
```

Belum ada dokumen escalation karena konfigurasi default hanya mematerialisasi Medium, High, dan Critical.

14.6 Tahap 3: snapshot kedua setelah sepuluh alert

Lima raw alert berikutnya masuk ke case yang sama. State bukan ditambah secara sembarang dari nilai sebelumnya; engine menghitung ulang snapshot berdasarkan raw alert yang terlihat untuk bucket tersebut. Hasilnya count 10, tiga IP unik, dan distribusi IP 5, 3, dan 2.

$$R_{10} = \frac{4 + 2 + 2}{3} = 2.67 \implies \text{Medium}$$

Potongan alarm_state final pada run kedua:

```
{
  "document": {"type": "alarm_state"},
  "alarm": {
    "id": "eccbdf4cc5236eff5993454b8c1ecd535e836e8bbab012d13bd196f9077bb347",
    "case_key": "coarse|002|5710|2026-08-26T10:00:00Z",
    "case_type": "auth",
    "bucket_start": "2026-08-26T10:00:00Z",
    "first_seen": "2026-08-26T10:05:11Z",
    "last_seen": "2026-08-26T10:18:03Z",
    "event_count": 10
  },
  "source_observed": {
    "srcip_unique_count": 3,
    "top_srcip": [
      {"value": "203.0.113.10", "count": 5},
      {"value": "198.51.100.24", "count": 3},
      {"value": "192.0.2.44", "count": 2}
    ]
  }
}
```

```

},
"risk": {
  "frequency_score": 2,
  "score": 2.67,
  "level": "Medium",
  "previous_level": "Low",
  "level_changed": true,
  "escalation_log_required": true
},
"source": {
  "index": "wazuh-alerts-4.x-2026.08.26",
  "raw_alert_count": 10,
  "sample_document_id": "dummy-5710-001"
}
}

```

14.7 Tahap 4: log alarm eskalasi

Karena risk level naik dari Low ke Medium, engine membuat satu dokumen immutable `alarm_escalation` pada index tanggal bucket yang sama:

```

{
  "_index": "siem-alarm-2026.08.26",
  "_id": "90c8d3535850c342651b62572c46c32fc8182cf8e2f49498a172e26c560595d4",
  "_source": {
    "document": {"type": "alarm_escalation"},
    "event": {
      "kind": "alert",
      "action": "risk_level_escalated"
    },
    "escalation": {
      "state_alarm_id": "eccbdf4cc5236eff5993454b8c1ecd535e836e8bbab012d13bd196f9077bb347",
      "level": "Medium",
      "previous_level": "Low",
      "reason": "risk_level_increased"
    }
  }
}

```

14.8 Apa yang terjadi pada source IP berbeda?

Contoh ini membuktikan perilaku default secara konkret:

- Source IP tidak di-replace oleh IP terakhir. Ketiganya tetap tampak dalam `srcip_samples` selama belum melewati batas sampel.
- Source IP tidak hilang dari perhitungan. `srcip_unique_count=3` menghitung kardinalitas exact untuk case ini.
- Frekuensi setiap IP dipertahankan pada `top_srcip`: 5, 3, dan 2.
- Sepuluh dokumen raw dan seluruh `full_log` tetap berada pada `wazuh-alerts-*`; state hanya menyimpan ringkasan bounded dan pointer `sample_document_id`.

- Jika jumlah IP melebihi `evidence_sample_limit`, hanya daftar sampel yang dipotong. Counter total raw tetap 10 pada contoh ini, dan raw evidence tetap tersedia. Untuk cardinality yang sangat besar, kapasitas counter in-memory tetap harus diuji pada data produksi.

Dengan demikian, kata “agregasi” berarti menggabungkan event ke satu alarm sambil mempertahankan bukti penting sebagai counter, unique count, sample, dan top value; bukan menimpa field source IP dengan nilai terbaru.

Bab 15

Contoh End-to-End: Failed Login

Raw alert pertama:

```
{
  "_index": "wazuh-alerts-4.x-2026.04.21",
  "_id": "qx3qrZ0BdT3L6MHNrDob",
  "_source": {
    "timestamp": "2026-04-21T02:42:05.483+0000",
    "agent": {
      "id": "002",
      "name": "srv-web-01",
      "ip": "192.168.100.99"
    },
    "rule": {
      "id": "5503",
      "level": 5,
      "description": "PAM: User login failed.",
      "groups": ["pam", "syslog", "authentication_failed"]
    },
    "decoder": {"name": "pam"},
    "data": {"dstuser": "kalime"}
  }
}
```

Inventory memberi Asset Value 4. Rule level 5 menjadi Threat Score 2. Case key:

```
coarse|002|5503|2026-04-21T02:00:00Z
```

Document ID state adalah SHA-256 penuh:

```
04edbcf067fe47b65ec7451c4e715dd391f369a916bb317ec3569b7509088495
```

Tabel 15.1: Perkembangan progressive alarm.

Count	A	T	F	Skor	Level
5	4	2	1	2.33	Low
55	4	2	3	3.00	Medium
510	4	2	5	3.67	High

Pada count 55 dibuat event Medium. Pada count 510 dibuat event High. State tetap memakai ID yang

sama dan hanya isinya yang diperbarui.

Bab 16

Dokumen alarm_state

Berikut bagian penting state pada transisi High:

```
{
  "timestamp": "2026-04-21T02:00:00Z",
  "document": {"type": "alarm_state"},
  "alarm": {
    "id": "04edbcf067fe47b65ec7451c4e715dd391f369a916bb317ec3569b7509088495",
    "case_key": "coarse|002|5503|2026-04-21T02:00:00Z",
    "deduplication_mode": "coarse",
    "case_type": "auth",
    "status": "open",
    "bucket_start": "2026-04-21T02:00:00Z",
    "bucket_size": "1h",
    "first_seen": "2026-04-21T02:42:05Z",
    "last_seen": "2026-04-21T02:58:33Z",
    "event_count": 510
  },
  "agent": {
    "id": "002",
    "name": "srv-web-01",
    "ip": "192.168.100.99"
  },
  "risk": {
    "asset_value": 4,
    "threat_score": 2,
    "frequency_count_1h": 510,
    "frequency_score": 5,
    "score": 3.67,
    "level": "High",
    "previous_level": "Medium",
    "level_changed": true,
    "escalation_log_required": true,
    "formula": "(A+B+C)/3"
  },
  "source": {
    "index": "wazuh-alerts-4.x-2026.04.21",
    "raw_alert_count": 510,
    "sample_document_id": "qx3qrZ0BdT3L6MHNrDob"
  },
  "soc": {
    "recommended_action": "Investigate",
    "sla": "1 hour",
    "notification": true,
  }
}
```

```
    "escalation_log": true
  }
}
```

Field `timestamp` pada state adalah awal bucket agar kartu alarm berada pada jam agregasinya. `first_seen` dan `last_seen` menyimpan rentang event. Raw evidence direferensikan melalui sample index dan sample document ID; dokumen raw lengkap tidak disalin ke destination.

Bab 17

Dokumen alarm_escalation

Ketika Medium naik menjadi High, event berikut dibuat dengan action create:

```
{
  "timestamp": "2026-04-21T02:58:33Z",
  "document": {"type": "alarm_escalation"},
  "event": {
    "kind": "alert",
    "category": ["siem_alarm"],
    "type": ["change"],
    "action": "risk_level_escalated",
    "created": "<waktu-proses-UTC>"
  },
  "escalation": {
    "id": "66e7ac65574a5ed439e62cdca0be239c07e90274c5cecf96be993d34bcd0529d",
    "state_alarm_id": "04edbcf067fe47b65ec7451c4e715dd391f369a916bb317ec3569b7509088495",
    "level": "High",
    "previous_level": "Medium",
    "reason": "risk_level_increased"
  },
  "risk": {
    "score": 3.67,
    "level": "High",
    "previous_level": "Medium"
  },
  "source": {"raw_alert_count": 510},
  "soc": {"notification": true, "escalation_log": true}
}
```

ID event dibentuk dari hash `escalation|alarm.id|level`. Dengan level default Medium, High, dan Critical, maksimum normal adalah tiga event per alarm per bucket. Create conflict HTTP 409 diperlakukan sebagai event sudah pernah dibuat. Hal ini memberikan at-most-one materialization per level walaupun run diulang.

`alarm_escalation` menyimpan snapshot count pada saat transisi. Karena itu event Medium dapat memiliki count 55 sementara state akhir mempunyai count 510. Perbedaan tersebut benar dan berguna untuk menjelaskan kapan ambang dilewati.

Bab 18

Lifecycle State dan Riwayat Level

State awal dibuat dengan `previous_level=null`. Jika level awal termasuk daftar eskalasi, event dengan reason `initial_eligible_level` dibuat. Bila level meningkat kemudian, reason menjadi `risk_level_increased`.

Riwayat pada contoh:

```
"level_history": [
  {"level": "Low",    "at": "2026-04-21T02:42:05Z"},
  {"level": "Medium", "at": "2026-04-21T02:50:10Z"},
  {"level": "High",   "at": "2026-04-21T02:58:33Z"}
]
```

Saat snapshot final ditulis, `alarm.status` berubah menjadi `finalized`. Jika risiko tetap High, update final tersebut memiliki `previous_level=High`, `level_changed=false`, dan tidak membuat event baru. Transisi sebelumnya tetap dapat dilihat pada `level_history` dan dokumen escalation immutable.

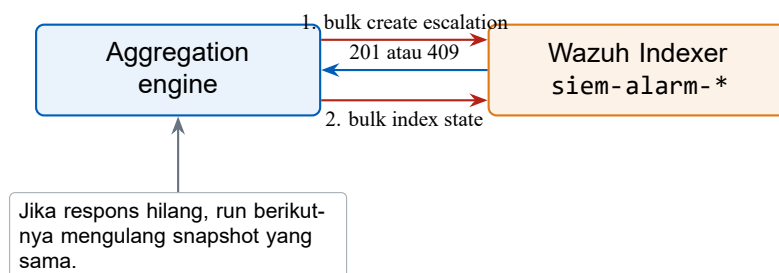
Penurunan level tidak dianggap eskalasi. Dalam alur normal count meningkat sehingga frequency tidak turun. Penurunan dapat terjadi bila raw data dihapus, scoring config diubah, atau backfill memakai sumber yang berbeda; keadaan tersebut harus diperlakukan sebagai perubahan administrasi yang perlu audit.

Bab 19

Two-Phase Bulk dan Idempotensi

Penulisan dilakukan dalam dua fase:

1. buat seluruh `alarm_escalation` yang diperlukan dengan `bulk create`;
2. setelah event terkait diterima atau terbukti sudah ada, tulis `alarm_state` dengan `bulk index`.



Gambar 19.1: Urutan penulisan agar event tidak tertinggal di belakang state.

Bulk response diperiksa per item, bukan hanya status HTTP. Status 429, 502, 503, dan 504 diulang dengan exponential backoff dan jitter. Hanya item yang gagal yang diulang. Payload NDJSON dibekukan sehingga retry mengirim snapshot yang sama.

Batch dibatasi oleh jumlah action dan byte UTF-8. Dokumen tunggal yang lebih besar dari batas menyebabkan run gagal sebelum write pertama. Ini mencegah keberhasilan parsial yang sudah dapat diprediksi sejak awal.

Bab 20

Checkpoint, Catch-Up, dan Late Alert

Checkpoint menyimpan akhir bucket tertutup terakhir yang selesai, hash identitas case, hash konfigurasi scoring, waktu update, dan metrik run. File ditulis ke temporary file, di-flush, diubah permission menjadi 0600, lalu dipindahkan secara atomik.

Checkpoint bukan counter dan bukan sumber kebenaran alarm. Raw index tetap menjadi sumber kebenaran. Fungsinya hanya mengatur recovery kalender agar outage tidak menimbulkan query storm.

Pada bootstrap, engine hanya mengambil bucket finalized terbaru. Pada outage, jumlah closed bucket yang diproses per run dibatasi, default dua. Bila source index untuk bucket yang diperlukan tidak tersedia, run gagal dan checkpoint tidak maju. Operator harus menyelidiki retensi atau menjalankan backfill terkontrol.

Destination berbeda dari source. Pada run non-kosong pertama dan rollover tanggal UTC, concrete daily destination memang belum ada. OpenSearch 2.19 dapat mengembalikan `_mget` HTTP 200 dengan satu `index_not_found_exception` per ID tanpa field status.[7, 8] Engine menerima kondisi bootstrap ini hanya ketika seluruh item batch menyebut `error.type` dan `error.index` yang tepat untuk concrete index yang diminta. Seluruh state dianggap baru dan two-phase bulk pertama membuat index melalui template yang telah terpasang. Respons campuran, index berbeda, status bertentangan, atau flip ada/hilang pada retry batch yang sama tetap gagal tertutup. Delete eksternal di antara batch tidak dapat dibuat atomik oleh API; operasi itu tidak didukung dan prosedur stop-before-delete tetap wajib.

Manual backfill harus memakai waktu UTC yang aligned dengan boundary:

```
--from 2026-04-21T02:00:00Z --to 2026-04-21T03:00:00Z
```

Backfill manual tidak memajukan checkpoint kalender. Timer dan service aktif harus dihentikan agar tidak ada run dengan lock berbeda atau konfigurasi berbeda.

Pokok penting. Late alert setelah finalisasi tidak otomatis memperbarui state. Ukur lag Filebeat ke Indexer dan pertahankan p99 jauh di bawah delay finalisasi. Jika SLO tersebut tidak dapat dicapai, naikkan delay atau jadwalkan prosedur reconciliation.

Bab 21

Fail-Closed dan Guardrail

Engine memilih gagal tertutup ketika menemukan keadaan yang dapat membuat count salah. Error sebelum bulk pertama tidak menulis destination apa pun. Error pada batch/fase write yang lebih akhir dapat meninggalkan write deterministik yang sudah diterima Indexer, tetapi checkpoint tidak maju; ID deterministik, create-only escalation, dan rerun membuat state konvergen tanpa menggandakan event. Contohnya:

- timestamp, agent ID, rule ID, atau rule level wajib tidak valid;
- shard search gagal atau search timeout;
- tidak ada concrete daily index yang ter-resolve;
- total hit tidak eksak atau scroll berhenti sebelum seluruh hit diterima;
- alert per bucket melewati `max_alerts_per_bucket`;
- case unik melewati `max_cases_per_bucket`;
- response `_mget` atau `_bulk` malformed;
- asset inventory stale atau tidak valid;
- checkpoint rusak atau hash identitas berbeda.

Missing daily destination bukan error apabila predicate bootstrap ketat di atas terpenuhi. Kondisi itu dicatat sebagai INFO karena normal sekali per tanggal UTC. Jika pesan yang sama muncul untuk index yang sebelumnya ada, operator harus menganggapnya indikasi delete/race, menghentikan timer, dan menjalankan audit serta controlled backfill. Membuat ulang index dengan checkpoint lama tidak mengembalikan bucket historis yang telah dilewati.

Default cap 100.000 raw alert dan 20.000 case per bucket adalah ceiling keselamatan, bukan kapasitas yang dijanjikan. Membagi bucket di tengah setelah cap tercapai akan menghasilkan frequency salah, sehingga engine menolak pendekatan tersebut. Initial scroll menggunakan `track_total_hits=true` karena Wazuh Indexer 4.14.7 menolak threshold numerik pada scroll context. Indexer karena itu menghitung total hit eksak sebelum cap diperiksa. Cap tetap mencegah pagination, agregasi, bulk, dan checkpoint untuk bucket yang terlalu besar, tetapi biaya exact-hit count tetap harus diukur dalam load test.

Process lock berbasis `fcntl` mencegah dua instance Linux yang memakai lock file sama berjalan bersamaan. `Systemd TimeoutStartSec=240s` membatasi eksekusi `Type=oneshot` agar run tidak

bertumpuk dengan jadwal berikutnya.

Bab 22

Keamanan dan Least Privilege

Koneksi ke Indexer wajib HTTPS dan verifikasi CA tidak dapat dimatikan melalui config produksi. URL harus cocok dengan Subject Alternative Name sertifikat. Runtime username **admin** serta password inline ditolak.

Role runtime yang disarankan:

Tabel 22.1: Hak minimum logis runtime.

Scope	Pattern	Hak
Cluster	–	cluster_composite_ops_ro, clear-scroll, dan envelope bulk yang dibutuhkan.
Source	wazuh-alerts-*	Read-only.
Destination	siem-alarm-*	Read, index, dan create-index.
Tenant	–	Tidak diperlukan.
Security/template/ISM	–	Tidak diberikan kepada runtime.

Template dan ISM dipasang satu kali menggunakan administrator, kemudian `install_template=false`. Installer membuat service user tanpa login shell, menolak managed symlink, membatasi filesystem melalui `ProtectSystem=strict`, menghapus Linux capabilities, mengaktifkan `NoNewPrivileges`, dan memberikan write path hanya untuk log serta state.

Asset inventory root-owned lebih dipercaya daripada payload decoded. Label Wazuh resmi hanya fallback. Pendekatan ini mengurangi risiko endpoint memalsukan nilai aset melalui isi log yang dikirim.

Bab 23

Performa dan Kapasitas

Current bucket dihitung ulang setiap lima menit. Dengan laju relatif merata dan 12 run per jam, total scan current mendekati 5,5 kali jumlah alert satu bucket. Finalisasi closed bucket menambah kira-kira satu kali, sehingga steady-state sekitar 6,5 kali jumlah alert per jam.

Jika satu bucket mengandung A raw alert, estimasi dokumen yang dibaca per jam:

$$\text{scan per jam} \approx 6.5A$$

Untuk $A = 20.000$, sekitar 130.000 raw document dibaca sepanjang satu jam, meskipun state akhir hanya mungkin beberapa ratus case. Penggunaan `_mget` dan bulk mengurangi round trip per case, tetapi tidak menghapus biaya scan snapshot.

Faktor kapasitas utama:

- peak EPS dan burst di dalam bucket;
- jumlah rule/case unik;
- jumlah source IP, URL, user, dan file unik yang harus dicounter;
- heap, disk latency, dan shard pressure pada Indexer AIO;
- ukuran hasil JSON dan jumlah event escalation;
- waktu retry ketika Indexer menghasilkan 429.

Kriteria load test yang disarankan adalah traffic minimal dua kali peak aktual, p95 run di bawah 60 detik, maksimum di bawah 240 detik, tanpa 429 atau failed shard, dan tanpa menyentuh batas memory systemd.

Bab 24

Dashboard dan Query Operasional

Index pattern dashboard adalah `siem-alarm-*` dengan time field `timestamp`. Karena state dan escalation berada pada index yang sama, semua visualisasi wajib memfilter `document.type`.

Query state alarm terbaru:

```
POST siem-alarm-*/_search
{
  "size": 50,
  "query": {
    "bool": {
      "filter": [
        {"term": {"document.type": "alarm_state"}},
        {"terms": {"risk.level": ["High", "Critical"]}}
      ]
    }
  },
  "sort": [{"alarm.last_seen": {"order": "desc"}}]
}
```

Query event yang dapat dikonsumsi aplikasi eksternal:

```
POST siem-alarm-*/_search
{
  "size": 100,
  "query": {
    "bool": {
      "filter": [
        {"term": {"document.type": "alarm_escalation"}},
        {"range": {"event.created": {"gt": "<cursor-terakhir>"}}}
      ]
    }
  },
  "sort": [
    {"event.created": "asc"},
    {"escalation.id": "asc"}
  ]
}
```

Panel minimum mencakup total state per risk level, top agent, top rule, alarm High dan Critical terbaru, top source IP, status open versus finalized, serta jumlah escalation per waktu. Jangan menjumlahkan `alarm.event_count` tanpa filter state karena event escalation membawa snapshot alarm yang sama.

Bab 25

Validasi Count terhadap Raw Evidence

Validasi satu alarm dilakukan menggunakan identitas persisnya. Untuk contoh case agent 002, rule 5503, bucket 02:00Z:

```
POST wazuh-alerts-4.x-2026.04.21/_count
{
  "query": {
    "bool": {
      "filter": [
        {"term": {"agent.id": "002"}},
        {"term": {"rule.id": "5503"}},
        {"range": {
          "timestamp": {
            "gte": "2026-04-21T02:00:00Z",
            "lt": "2026-04-21T03:00:00Z"
          }
        }}
      ]
    }
  }
}
```

Nilai `count` harus sama dengan tiga count pada `alarm_state`. Untuk mode `target-aware`, tambahkan filter `destination` sesuai case key. Untuk `file-aware`, tambahkan `path` yang sama.

Validasi juga evidence:

- cardinality source IP harus sama dengan `srcip_unique_count`;
- terms aggregation source IP harus cocok dengan `top_srcip`;
- earliest dan latest timestamp harus cocok dengan `first/last seen`;
- source sample document ID harus benar-benar ada;
- asset source dan value harus sesuai inventory yang disetujui.

Perbedaan count tidak boleh langsung diperbaiki dengan mengedit state. Periksa timezone index, filter rule, late alert, perubahan inventory/config, dan status finalisasi; kemudian lakukan replay dari raw evidence bila diperlukan.

Bab 26

Instalasi dan Urutan Go-Live

Urutan implementasi yang aman:

1. verifikasi versi paket dan semua service Wazuh aktif;
2. verifikasi certificate chain, SAN, dan Filebeat output;
3. jalankan seluruh unit test serta validator JSON;
4. jalankan installer; pastikan timer tetap disabled;
5. buat role dan user Indexer khusus;
6. isi secret runtime dan inventory aset sebenarnya;
7. pasang index template dan ISM policy menggunakan administrator;
8. jalankan field audit pada traffic representatif;
9. lakukan manual run, cek journal, lalu query state dan escalation;
10. cross-check beberapa case terhadap raw alert;
11. jalankan shadow/load test pada peak traffic;
12. baru aktifkan timer dan monitor dua sampai tiga run pertama.

Installer sengaja tidak mengaktifkan timer pada instalasi baru ataupun upgrade. Jika timer sebelumnya enabled, installer menghentikan dan menonaktifkannya agar operator melakukan validasi pasca-upgrade.

Index template harus ada sebelum write pertama agar daily index pertama menerima mapping yang benar. ISM template hanya otomatis melekat pada index baru yang cocok; index lama perlu diperiksa secara terpisah sebelum diberi policy.

Bab 27

Monitoring Harian dan Troubleshooting

Operator sebaiknya memantau:

- hasil dan durasi unit `systemd`;
- 429, timeout, shard failure, cap exceeded, dan malformed response;
- waktu update checkpoint;
- jumlah state open/finalized;
- persentase `asset.source=default`;
- p95/p99 lag raw alert ke Indexer;
- CPU, memory, heap, dan disk latency host AIO;
- adanya destination daily index dan status ISM.

Gejala	Pemeriksaan awal
Tidak ada alarm	Pastikan raw daily index ada, window benar, source filter tidak mengecualikan semuanya, dan refresh destination sudah terjadi.
Count lebih kecil	Periksa late alert, timestamp/index date, field wajib malformed, atau snapshot belum final.
Count lebih besar	Pastikan query pembanding menggunakan batas <code>gte/lt</code> dan filter case key yang sama.
Asset default	Lengkapi inventory, periksa agent ID/name, dan audit re-enrollment.
Service timeout	Ukur EPS/cardinality dan Indexer pressure; jangan langsung menaikkan cap.
Checkpoint mismatch	Ada perubahan identitas case; gunakan shadow prefix atau controlled migration.
Escalation tampak ganda	Filter berdasarkan <code>escalation.id</code> ; create-only seharusnya menghasilkan satu ID per level.

Gejala	Pemeriksaan awal
<code>_mgetindex_not_found</code>	Normal pada first nonempty run atau rollover bila log menyebutkan bootstrap. Jika index sebelumnya ada, hentikan timer dan audit delete/backfill; jangan menghapus checkpoint.

OnFailure saat ini mencatat pesan critical ke journald, tetapi tidak otomatis mengirim email, webhook, atau tiket. Integrasi notifikasi eksternal perlu dibangun dari event `alarm_escalation` atau monitoring systemd.

Bab 28

Batas Desain dan Tuning

Default coarse adalah keputusan pengurangan noise, bukan kebenaran universal. Satu rule failed login pada satu server dapat menyatukan banyak username dan source IP. Ini berguna untuk melihat campaign terhadap host, tetapi tidak menghasilkan satu alarm per akun. Mode `user_aware` belum tersedia.

Demikian pula, sensor Suricata terpusat dapat menyatukan rule yang sama untuk banyak destination. Gunakan override `target_aware` atau `target_port_aware`. Rule FIM yang memakai ID sama untuk banyak file umumnya lebih mudah dianalisis dengan `file_aware`.

Observed evidence dibatasi ketika dimaterialisasi, tetapi counter internal tetap melihat semua nilai unik di snapshot. Cardinality ekstrem dapat menggunakan memory besar sebelum dokumen diringkas. Cap alert, cap case, dan MemoryMax menjadi lapisan pertahanan; hasil load test menentukan apakah rule noisy perlu dikecualikan.

Skor rata-rata tiga komponen tidak memasukkan business impact selain asset value, control effectiveness, exposure, threat intelligence, confidence, atau analyst feedback. Tuning skor harus dimulai dari kebijakan tertulis dan contoh kasus, lalu diuji menggunakan shadow destination. Jangan mengubah threshold langsung pada index produksi tanpa rencana backfill dan interpretasi dashboard baru.

Bab 29

Checklist Penerimaan Produksi

Gunakan daftar berikut sebagai keputusan go/no-go:

- ☐ Paket Wazuh dan Filebeat sama dengan baseline yang direview.
- ☐ TLS berhasil tanpa insecure mode; URL cocok dengan SAN.
- ☐ Runtime bukan admin dan role lulus uji read/mget/scroll/bulk/create-index.
- ☐ Template dan ISM terpasang sebelum destination index dibuat.
- ☐ Inventory seluruh agent penting lengkap; asset default tidak boleh muncul tanpa rencana.
- ☐ Field audit mencakup custom decoder dan rule produksi.
- ☐ `bucket_minutes=60` dan index date terbukti mengikuti UTC.
- ☐ Minimal satu state dan satu escalation aktual sudah diverifikasi.
- ☐ Tiga count state sama dan cocok dengan query raw evidence.
- ☐ Current state berkembang dan closed bucket menjadi finalized.
- ☐ Late-alert SLO berada di bawah finalization delay.
- ☐ Failure injection membuktikan retry tidak menduplikasi event.
- ☐ Load test dua kali peak memenuhi p95 dan TimeoutStartSec.
- ☐ Checkpoint owner/mode benar dan bergerak hanya setelah run sukses.
- ☐ Dashboard memfilter document type dengan benar.
- ☐ Monitoring service failure dan prosedur backfill sudah dimiliki operator.
- ☐ Rollback window, backup, dan PIC keputusan sudah ditetapkan.

Go-live dinyatakan berhasil setelah beberapa bucket aktual melewati lifecycle open, escalation, dan finalized tanpa count drift, late-alert loss, atau degradasi Indexer.

Bab 30

Kesimpulan

Sistem ini membentuk lapisan agregasi yang dapat dijelaskan: satu identitas case, satu state deterministik, bukti observed yang diringkas, dan event escalation yang immutable per level. Raw Wazuh tidak disentuh, sehingga investigasi tetap dapat kembali ke dokumen asal.

Untuk baseline satu jam, rumus dan implementasi konsisten. Source IP berbeda diagregasikan sebagai counter dan sampel, bukan di-replace. Progressive snapshot menghindari double count, sedangkan two-phase bulk, deterministic ID, lock, dan checkpoint menyediakan recovery yang aman.

Nilai praktis sistem bergantung pada tiga disiplin: memilih mode deduplikasi per rule, memelihara inventory aset, dan mengukur lag serta kapasitas pada traffic nyata. Tanpa ketiganya, agregasi tetap berjalan tetapi dapat terlalu kasar, salah prioritas, atau terlambat memfinalisasi bukti.

Keputusan yang direkomendasikan adalah menjalankan shadow deployment terlebih dahulu, mempertahankan bucket 60 menit, memperbaiki contoh dokumentasi agar menjadi golden test, dan memberikan status production hanya setelah checklist penerimaan terpenuhi.

Lampiran A

Ringkasan Konfigurasi Baseline

```
{
  "opensearch_url": "https://127.0.0.1:9200",
  "username": "siem_alarm_service",
  "password_env": "WAZUH_PASS",
  "verify_ssl": true,
  "ca_cert": "/opt/wazuh-risk-scoring/root-ca.pem",
  "source_index": "wazuh-alerts-4.x-{date}",
  "destination_index_prefix": "siem-alarm",
  "deduplication_mode": "coarse",
  "bucket_minutes": 60,
  "process_current_bucket_only": true,
  "lookback_overlap_minutes": 7,
  "escalation_log_enabled": true,
  "escalation_log_levels": ["Medium", "High", "Critical"],
  "threat_level_strategy": "max",
  "max_alerts_per_bucket": 100000,
  "max_cases_per_bucket": 20000,
  "page_size": 1000,
  "mget_batch_size": 1000,
  "bulk_max_actions": 1000,
  "bulk_max_bytes": 5242880,
  "max_catchup_buckets_per_run": 2,
  "install_template": false
}
```

Key lain seperti timeout, retry, evidence limits, exclusions, log path, lock path, assets path, dan checkpoint path harus dipertahankan dari config lengkap. Cuplikan ini adalah baseline penjelasan, bukan pengganti file proyek.

Lampiran B

Matriks Field Output

Field	Makna
<code>document.type</code>	Memisahkan alarm state dan escalation event.
<code>alarm.id</code>	SHA-256 case key dan document ID state.
<code>alarm.case_key</code>	Identitas kasus yang dapat diaudit.
<code>alarm.status</code>	Lifecycle bucket, bukan workflow analis.
<code>alarm.first_seen / last_seen</code>	Batas timestamp event dalam snapshot.
<code>alarm.event_count</code>	Jumlah raw document case.
<code>rule.level</code>	Level terpilih menurut strategy.
<code>rule.level_counts</code>	Distribusi level pada snapshot.
<code>source_observed.top_srcip</code>	Source IP paling sering beserta count terbatas.
<code>target_observed.*</code>	Destination, port, dan protocol yang diamati.
<code>entity_observed.*</code>	User, URL, user agent, file, hash, CVE, dan SCA.
<code>asset.*</code>	Nilai, kategori, owner, environment, dan sumber metadata aset.
<code>risk.score / level</code>	Skor rata-rata dan kategori hasil.
<code>risk.previous_level</code>	Level state sebelum update saat ini.
<code>risk.level_history</code>	Riwayat kenaikan level di dalam bucket.
<code>source.raw_alert_count</code>	Counter raw yang harus sama dengan event count.
<code>source.sample_document_id</code>	Pointer ke raw sample paling awal secara deterministik.
<code>soc.recommended_action / sla</code>	Saran tindakan berbasis risk level.
<code>escalation.state_alarm_id</code>	Relasi event immutable ke state.
<code>event.created</code>	Waktu event escalation dimaterialisasi.

Lampiran C

Glosarium

AIO	All-in-one; manager, indexer, dan dashboard berada pada satu host.
Bucket	Rentang waktu tetap untuk membentuk identitas dan frequency count.
Case key	String deterministik yang mendefinisikan satu alarm agregat.
Checkpoint	Penanda recovery bucket tertutup yang sudah selesai.
Current bucket	Bucket waktu yang masih berjalan dan dihitung ulang.
Closed bucket	Bucket yang boundary akhirnya telah lewat.
Finalized	Snapshot closed bucket telah berhasil ditulis.
Evidence	Nilai pendukung seperti IP, user, URL, port, file, atau CVE.
Idempotent	Operasi aman diulang tanpa membuat hasil logis ganda.
Late alert	Alert yang baru terlihat di Indexer setelah finalisasi bucket.
Raw alert	Dokumen deteksi asli pada <code>wazuh-alerts-*</code> .
Shadow run	Menjalankan engine ke destination terpisah tanpa menjadi sumber operasi utama.
State	Representasi terbaru satu kasus dalam bucket.
Escalation	Event immutable ketika level eligible pertama muncul atau meningkat.

Lampiran D

Referensi Teknis

Referensi

- [1] Wazuh, *Wazuh 4.14.7 Release Notes*.
<https://documentation.wazuh.com/current/release-notes/release-4-14-7.html>
- [2] Wazuh, *Wazuh Indexer Indices*.
<https://documentation.wazuh.com/current/user-manual/wazuh-indexer/wazuh-indexer-indices.html>
- [3] Wazuh, *Rules Classification*.
<https://documentation.wazuh.com/current/user-manual/ruleset/rules/rules-classification.html>
- [4] Wazuh, *Wazuh 4.14.7 default SSHD rules, 0095-sshd_rules.xml*.
https://github.com/wazuh/wazuh/blob/v4.14.7/ruleset/rules/0095-sshd_rules.xml
- [5] Wazuh, *Agent Labels*.
<https://documentation.wazuh.com/current/user-manual/agent/agent-management/labels.html>
- [6] OpenSearch, *Bulk API*.
<https://docs.opensearch.org/latest/api-reference/document-apis/bulk/>
- [7] OpenSearch, *Multi-get Documents API*.
<https://docs.opensearch.org/latest/api-reference/document-apis/multi-get/>
- [8] OpenSearch 2.19, *MultiGetResponse.java: failure serialization*.
<https://github.com/opensearch-project/OpenSearch/blob/2.19/server/src/main/java/org/opensearch/action/get/MultiGetResponse.java>
- [9] OpenSearch, *Index State Management Policies*.
<https://docs.opensearch.org/latest/im-plugin/ism/policies/>
- [10] Proyek SIEM Alarm, *README.md, final_checklist_siem_alarm_wazuh_4_14_7.md*, dan source code yang berada pada repository yang sama dengan dokumen ini.